



CabRoster

Design decisions & critical edge cases

Why the solution is shaped the way it is, and everything that must be validated before the code is done.

Agenda

1

Architecture

Layers, folders and the test contract

3

Auth & access

Tokens, 401 vs 403 vs 404

5

Business rules

Booking, cancel, trip, assign, driver

7

Errors

The status-code decision matrix

2

Data model

Snapshots, soft deletes, derived counts

4

Time

Fake clock and the exact boundaries

6

Lists

Filtering, sorting, pagination

8

Quality

Test strategy and the final checklist

01

Architecture

Thin routers, rich services, one source of time

Request flow: four layers

Each layer has one job. Business rules live in exactly one place.

Router

`routers/*.py`

Parse input, pick dependencies (user, db, now), call a service, map to schema

Dependencies

`core/deps.py`

`get_current_user`, `require_roles`, `get_now`, `PageParams`

Service

`services/*.py`

All BR-xx rules. Raises `HTTPException` with the right code

Model / DB

`models.py`, `database.py`

Tables, enums, relationships. No rules here

Why: a rule written once in a service is enforced the same way from every endpoint, and can be unit-tested without HTTP.

The Test Contract: fixed plug points

Tests swap in a fresh DB and a fake clock through these names

Hook	How the tests use it	If you break it
app.main.app	TestClient(app)	Nothing runs
database.Base / SessionLocal / get_db	In-memory SQLite per test; SessionLocal re-bound	Tests hit your real DB file
core.deps.get_now	dependency_overrides → fake IST clock	Every cut-off test fails
core.security.hash_password	Creates the first ADMIN	Every admin test errors
models.User / Role	Inserts ADMIN row directly	Every admin test errors
config.settings.JWT_SECRET_KEY / ALGORITHM	Tests forge expired / foreign tokens	Token tests fail

JWT exp uses the REAL clock. If it used get_now(), tokens made "on 25-Sep-2026" would already be expired when the suite runs later.

02

Data model

Four tables, and the few choices that matter

Five data decisions

Each one prevents a class of bugs



Snapshot address & zone on booking

Profile edits must not move an existing booking to another zone after trips are planned.



Soft delete only

Cancel = status. Deactivate = is_active false. History and reports stay correct.



seats_booked is computed, not stored

Count non-cancelled bookings on the trip. A stored counter drifts on every missed update.



trip_id nullable on booking

BOOKED = no trip yet. Cancel, remove or trip-cancel all reset trip_id to NULL.



Enums for role, zone, slot, statuses

Invalid values are rejected automatically with 422, and the DB never holds a typo.

Uniqueness: DB constraint or service check?

Some rules can't be a plain unique index

Rule	Where	Why
email, employee_code, license_no, registration_no unique	DB unique + service check (409)	Always true, no exceptions
One active booking per employee/date/slot (BR-03)	Service query	A CANCELLED booking must not block a new one
One trip per cab / driver per date+slot (BR-07)	Service query	A CANCELLED trip must free the cab and driver
Seats $\leq$ capacity (BR-09)	Service, inside the assign transaction	Depends on the counts across many rows

E-mail and registration numbers are normalised (lower / upper case) BEFORE the uniqueness check, otherwise "A@x.in" and "a@x.in" both get in.

03

Authentication & access

Who are you, what may you do, and what may you see

Token design

OAuth2 password flow → HS256 JWT

Payload

```
{
  "sub": "17",
  "role": "EMPLOYEE",
  "exp": 1790000000
}
```

Login is form data (username = e-mail), not JSON. E-mail match is case-insensitive.

- `get_current_user` loads the user from the DB on EVERY request
- Role comes from the DB row. The role claim is informational only; a forged role=ADMIN token must still get 403
- Missing user or `is_active = false` → 401 (a deactivated user's old token dies immediately)
- Any decode problem (bad signature, expired, no sub, garbage) → 401 with WWW-Authenticate: Bearer
- `hashed_password` never appears in any response

401 vs 403 vs 404

The order of checks decides the status code

1	Authenticate	No / bad / expired token, inactive user	401
2	Authorise role	Role not allowed on this endpoint, checked BEFORE loading the object	403
3	Load & own	Object missing, or not yours (employee booking, driver trip)	404
4	Validate & act	Input or state rules	409 / 422

Why 404 for "not yours": a 403 would confirm the ID exists, which lets someone probe other people's bookings and trips.

Permission matrix

The full suite calls every endpoint with every role: 26 × 401 checks and 46 × 403 checks

Endpoint group	ADMIN	EMPLOYEE	DRIVER
/auth/me (GET, PATCH)	Yes	Yes	Yes (PATCH: phone only)
/users/*, /cabs/*, /trips/*, GET /bookings	Yes	403	403
POST /bookings, GET /bookings/me	403	Yes	403
GET /bookings/{id}, PATCH .../cancel	Yes, any	Own only (else 404)	403
/driver/trips/*	403	403	Own only (else 404)
/reports/daily-summary (stretch)	Yes	403	403

04

Time

The most common source of hidden bugs

One clock: get_now()

Inject time, never read it

Untestable

```
def create_booking(...):  
    if datetime.now() >= cutoff:  
        ...
```

Testable

```
def create_booking(...,  
    now = Depends(get_now)):  
    if now >= cutoff: ...
```

- `get_now()` returns a timezone-aware IST datetime (UTC+05:30). Compare aware with aware; convert with `to_ist()` if needed
- Deadlines are computed, not hard-coded: `departure(date, slot) - 60 min`, etc. All times live in `config.py`
- "Before the cut-off" = `now < cutoff`. Exactly on the second counts as closed
- PICKUP cut-off is 21:00 on the previous CALENDAR day: a Monday pickup closes Sunday 21:00, even though Sunday has no service

Every time boundary the tests probe

Travel date Monday 28-Sep-2026

Action	Last OK moment	First refused moment	Refused with
Book Mon PICKUP	Sun 20:59:59	Sun 21:00:00	422
Book Mon DROP	Mon 13:59:59	Mon 14:00:00	422
Book Tue DROP at Mon 15:00	allowed	Mon's 14:00 cut-off does not apply	—
Employee cancels Mon PICKUP	Mon 05:29:59	Mon 05:30:00	422
Employee cancels Mon DROP	Mon 15:29:59	Mon 15:30:00	422
Admin cancels an assigned booking	any time trip is SCHEDULED	once the trip is IN_PROGRESS	409
Driver starts PICKUP trip	from Mon 06:00:00	Mon 05:59:59, any other day	422
Driver starts DROP trip	from Mon 16:00:00	Mon 15:59:59	422
Booking horizon (now Mon 28)	Mon 05-Oct (+7)	Tue 06-Oct (+8), any past date	422

05

Business rules

Booking, cancellation, trips, assignment, driver flow

Create booking: checks in order

Put cheap, input-only checks first, then checks that need the database

1	BR-01 / BR-02	Weekday? Between today and today+7?	422
2	BR-05	Profile has home_address AND zone?	422
3	BR-04	now < booking cut-off for that slot?	422
4	BR-03	No other non-cancelled booking for employee + date + slot?	409
5	Snapshot	Copy zone + address from profile; status BOOKED; trip null	201

Edge cases: Sat/Sun for both slots · today's PICKUP (always closed) vs today's DROP before 14:00 · rebook after cancel · same employee different slot · deactivated employee (401) · invalid date "2026-02-30" (422).

Cancel booking

The same endpoint behaves differently for employee and admin

Case	Employee	Admin
Someone else's booking	404	allowed
Status BOOKED / ASSIGNED, before cut-off	200	200
After cancellation cut-off, trip still SCHEDULED	422	200
Its trip already IN_PROGRESS	409	409
Already CANCELLED / BOARDED / NO_SHOW / COMPLETED	409	409
Unknown id	404	404

On success: status CANCELLED, cancelled_at set, trip_id = NULL. The seat is free again and a new booking can be assigned to it.

Create trip

Order: date rules → existence (404) → eligibility (422) → clashes (409)

Case	Result
Saturday / Sunday / past / more than 7 days ahead	422
cab_id or driver_id does not exist	404
Cab is_active = false	422
driver_id is an EMPLOYEE or ADMIN, or an inactive DRIVER	422
Same cab already on a non-cancelled trip, same date + slot	409
Same driver already on a non-cancelled trip, same date + slot	409
Same cab / driver on a different slot or date	201
Previous trip for that cab + driver was CANCELLED	201 (cancelled trips don't count)

Assign bookings: all-or-nothing

Validate the whole list first, change rows only after every check passed

```
if trip.status != SCHEDULED: 409
for id in booking_ids:
    b = get(id)                # 404
    b.status == BOOKED        # else 409
    same date/slot/zone       # else 409
used = seats_booked(trip)
if used + len(ids) > capacity: 409
# only now mutate
for b in bookings:
    b.trip_id = trip.id
    b.status = ASSIGNED
commit()
```

- Capacity counts seats ALREADY on the trip: 2 assigned + 3 new on a 4-seat cab → 409
- Exactly full (4 of 4) is fine. seats_available becomes 0
- [good, good, wrong-zone] → 409 and the two good ones are still BOOKED
- Empty list or duplicate ids → 422 (schema)
- Booking already on another trip → 409; cancelled booking → 409
- Trip CANCELLED / IN_PROGRESS / COMPLETED → 409

Remove, cancel trip, and capacity changes

Undo paths are where hidden state bugs appear

Operation	Must happen	Must be refused
Remove booking from trip	Booking → BOOKED, trip_id NULL; seats drop	Not on this trip (404); trip not SCHEDULED (409)
Cancel trip	Trip CANCELLED; active bookings → BOOKED, trip_id NULL	Trip IN_PROGRESS / COMPLETED / CANCELLED (409)
... with a cancelled passenger	That booking stays CANCELLED (not revived)	—
Reduce cab capacity	OK if $\geq$ seats on each SCHEDULED trip	Below seats on a SCHEDULED trip (409)
... trip already COMPLETED	OK: finished trips don't count	—
Deactivate cab	Hidden from available_on; cannot start new trips	New trip with it (422)

Driver flow

Ownership first (404), then state (409), then time (422)

Step	Allowed when	Refused
View / start / mark / complete	trip.driver_id == me	Other driver's or unknown trip → 404
Start	SCHEDULED, on travel date, now ≥ departure – 30 min	Wrong state 409 · outside window 422
Mark BOARDED / NO_SHOW	Trip IN_PROGRESS, booking on this trip, booking ASSIGNED	Not started 409 · not on trip 404 · marked already 409 · other value 422
Complete	IN_PROGRESS	SCHEDULED or COMPLETED → 409
On complete	BOARDED → COMPLETED · unmarked ASSIGNED → NO_SHOW · NO_SHOW stays	—
Empty trip (0 passengers)	Can start and complete normally	—

State machines as guard tables

Write the guard first, then the transition

Booking: from	to	by
—	BOOKED	employee create
BOOKED	ASSIGNED	admin assign
ASSIGNED	BOOKED	admin remove / trip cancel
BOOKED, ASSIGNED	CANCELLED	employee / admin
ASSIGNED	BOARDED, NO_SHOW	driver
BOARDED	COMPLETED	trip complete
ASSIGNED	NO_SHOW	trip complete

Trip: from	to	by
—	SCHEDULED	admin create
SCHEDULED	IN_PROGRESS	driver start
IN_PROGRESS	COMPLETED	driver complete
SCHEDULED	CANCELLED	admin cancel

Anything not in these tables → 409.

06

Lists

Filtering, sorting and pagination done safely

List design decisions

One helper pair (apply_sort + paginate) used by all six list endpoints

- 1 Whitelist sort fields** `sort_by: Literal[...]`. Anything else → 422. Never pass user text to `order_by` (e.g. `sort_by=hashed_password`).
- 2 Deterministic order** Always add `id ASC` as a tie-breaker, so pages never repeat or skip rows when values are equal.
- 3 Count the filtered query** `total = COUNT` over the filtered subquery, not the whole table and not the page.
- 4 Envelope maths** `pages = ceil(total/size)`; `total 0` → `pages 0`; page beyond the end → `items []` (not an error).
- 5 Limits** `page ≥ 1`, `1 ≤ size ≤ 100`, default size 10 → else 422.
- 6 Endpoint defaults** Ascending everywhere, except `/bookings/me` and `/driver/trips`: `travel_date DESC`.

Filter semantics

Every filter is optional; filters combine with AND

Filter	Exact meaning	Edge case
unassigned=true	trip_id IS NULL AND status = BOOKED	Cancelled bookings are NOT unassigned
date_from / date_to	Inclusive on both ends	date_from > date_to → 422
q (users)	ILIKE on full_name, email, employee_code	Case-insensitive, partial
q (bookings)	ILIKE on employee full_name, employee_code	Needs a join to users
q (cabs)	ILIKE on registration_no, model	—
available_on + slot	Active cabs with no non-cancelled trip then	Only one of the two → 422
Enum filters	role, zone, slot, status	Unknown value → 422 automatically
is_active	Boolean	"maybe" → 422

07

Errors & validation

One matrix, applied consistently

Status-code decision matrix

Ask the questions in this order

Question	If yes	Examples
Is the token missing, invalid, expired, or the user inactive?	401	No header · forged signature · deactivated
Is this role never allowed here?	403	Employee on /users · admin on POST /bookings
Does the object not exist, or belong to someone else?	404	Other's booking · other driver's trip
Is the input malformed or out of range?	422	Bad regex · capacity 13 · empty list · bad enum
Does a business rule about the input fail?	422	Weekend · past · deadline passed · no profile
Does it clash with existing data or current state?	409	Duplicate · full cab · wrong status · busy cab
Otherwise	200 / 201	Never 500: garbage ids → 422

Input validation rules

Pydantic does most of the work. Normalise first, then validate.

Field	Rule	Tested edge values
full_name	2–100 chars after trimming	Rejected: "A", 101 chars · Accepted: "A!", 100 chars
email	Valid, stored lower-case, unique case-insensitively	John.DOE@X.IN is stored as john.doe@x.in
phone	$^{[6-9]}\backslash d\{9\}\$$	Rejected: 9 or 11 digits, starts with 5, +91 · Accepted: starts with 6
password	≥ 8 chars, ≥ 1 letter, ≥ 1 digit	Rejected: "abcdef1", letters only · Accepted: "abcdefg1"
registration_no	Upper-case, then $^{[A-Z]\{2\}\backslash d\{2\}[A-Z]\{1,2\}\backslash d\{4\}\$$	Accepted: KA01A1234 · Rejected: KA-01-AB-1234, spaces
capacity	Integer 1–12	Rejected: 0, -1, 13, "four" · Accepted: 1, 12
zone / slot / status	Exact enum values (upper-case)	Rejected: "east", "MARS"
booking_ids	Non-empty, unique ints	Rejected: [], [5, 5]

08

Quality

How we know the solution is right

Test strategy

Two suites, same harness: fresh in-memory DB and a fake clock for every test

38

Simple suite

Given to students. 1–2 happy-path checks per API.

392

Full suite

Every rule, boundary second, role pair and error code.

5

Stretch

Daily-summary report (bonus).

46/48

Bugs caught

Bugs injected into the reference solution. The 2 survivors are on unreachable code paths.

Examples of injected bugs: using > instead of >= at a cut-off · not freeing the seat on cancel · partial assignment · trusting the token's role · floor instead of ceil for pages.

Definition of done: 16 checks

Verify each one yourself before you say "finished"

- ✓ All time via Depends(get_now)
- ✓ Role read from DB; inactive → 401
- ✓ Not-owned → 404, never 403
- ✓ Cut-offs use now < cutoff
- ✓ Cancel frees the seat (trip_id NULL)
- ✓ Capacity includes seats already taken
- ✓ Trip-cancel keeps CANCELLED bookings cancelled
- ✓ Sort whitelist + id tie-break
- ✓ JWT exp on the real clock
- ✓ Role check before object lookup (403)
- ✓ E-mail / registration normalised before unique checks
- ✓ PICKUP cut-off = previous calendar day 21:00
- ✓ Assignment validates all, then writes
- ✓ Cancelled trips / bookings never block
- ✓ Complete: ASSIGNED → NO_SHOW
- ✓ pages = ceil, total = filtered count

The three ideas behind the whole design

1

Inject what changes

Time, DB and user are dependencies, so they can be faked and tested.

2

Validate fully, then write

No half-done state: all-or-nothing assignment, guard-then-transition.

3

Never lose history

Soft deletes, snapshots, and statuses in place of deleted rows.